E-Graph-Augmented Tool-in-the-Loop Agent for High-Coverage RTL Verification

Chiyu Chen(cc5583), Mingyuan Zheng(mz3143), Yiwen Chen(yc4653)

COMS 6998: AI-Native Computing | Fall 2026

P1: Exploration & Progress | Due October 26, 2026, 11:59 PM ET

I. PROJECT DIRECTION AND UPDATES

Our question is whether bounded e-graph normalization of uncovered RTL predicates helps an LLM produce higher-coverage stimulus, or reach the same coverage with fewer model tokens and simulator executions, than raw coverage feedback. This tests the proposed loop: specification-guided generation, simulation, coverage extraction, semantic grouping, and refinement. Verification quality also requires a sound oracle; coverage alone cannot establish functional correctness.

Since the proposal, we have narrowed the first controlled experiment to JSON cycle actions consumed by a frozen SystemVerilog driver and specification-based checker. This prevents an LLM from changing the DUT or weakening assertions, and isolates the effect of feedback. It does not yet measure free-form testbench or assertion generation or CVDP pass@1.

The original project plan compared text-only A, raw coverage B, and semantic coverage C. Earlier arbiter, FIFO, and timer pilots exercised all three arms; the new held-out queue run focuses on C versus B, the primary contrast that isolates the proposed contribution. A remains necessary on harder designs to measure how much improvement comes from raw execution feedback alone.

II. EXPLORATION AND FINDINGS

Arbiter, FIFO, and lite-timer pilots validated the tool loop but saturated early: B and C reached their declared targets in the same first round on the FIFO and timer. In the earlier queue pilot with GPT-6.1 Sol, both arms reached 41/41 line points by round 3 in every repetition; C used more tokens. These observations motivated a held-out queue with 12 source-distinct read guards and 12 write guards, typed full activation conditions, source mapping, bounded egg saturation, and Z3 checks. Read and write cases remain separate equivalence classes.

We repeated the matched B/C pilot with the explicitly requested GPT-6 Sol model. Both arms receive the same specification, RTL, seed, checker logs, and raw DUT counters; C alone receives egg-derived equivalent-guard groups. Three independent repetitions per arm and RTL version each have four rounds, at most four new cycles per round. Each simulator execution replays the six-cycle seed and all accumulated actions; the predeclared target is 41/41 Verilator line/case points.

This evaluation follows the useful parts of CVDP and LLM4Cov: executable task validity, declared coverage thresholds, per-round progress, and failures retained rather than silently dropped. ROVER, ASPEN, SmoothE, and e-boost evaluate equivalence, optimization, or extraction quality; their PPA and extraction scores are not verification coverage gains. We therefore require both a proved semantic mapping and an independently measured downstream stimulus outcome.

TABLE I. GPT-6 SOL QUEUE PILOT (LIVE; B=RAW, C=RAW+EGG)

RTL	Arm	Target round (3 reps)	Final line / branch	Tokens to target
original	B	3, 4, 4	41/41; 14/14	74891
original	C	3, 4, 3	41/41; 14/14	73738
variant	B	3, 3, 3	41/41; 14/14	70272
variant	C	2, 3, 3	41/41; 14/14	66377

C reached the target one round earlier in 2 of 6 index-matched repetitions and tied in 4; B reached earlier in none. C minus B mean target cost was -1152 API tokens on the original and -3895 on the variant. The first-round C prompt itself cost about 1.5k more input tokens. Target savings mainly arise when a run reaches 41/41 one round earlier. All 12 runs still executed all four rounds; their final line/branch/toggle coverage was identical.

The apparent advantage is driven by one earlier C run per syntax version. Repetition labels organize independent, non-deterministic model sessions; identical seeds do not pair model outputs statistically. This one IP and two syntax variants cannot support a significance or cross-design generalization claim. The earlier GPT-6.1 Sol queue pilot showed no C advantage at the same target.

At fixed four-round budget, every repetition reached 41/41 line, 14/14 branch, and 344/408 toggle points. Thus the target-cost metric answers a different question from final coverage: how much resource was spent before the first threshold crossing, not how much the full experiment spent. Across all four rounds, C was more expensive in the original version and less expensive in the variant. We report both views and retain the full trajectory instead of choosing only the favorable stopping rule.

Toggle is not an interchangeable proxy for line coverage. It counts separate 0-to-1 and 1-to-0 transitions per signal bit, including bits that may be fixed under this configuration. The 64 unhit directions remain in the denominator until reachability is proved. Likewise, the two RTL versions have different expression-counter denominators, so expression percentages should only be compared within each frozen version. No source hit is inferred from an e-graph equivalence class.

III. IMPLEMENTATION AND INFRASTRUCTURE PROGRESS

The repository now has version-locked queue DUTs, an independent occupancy/data checker, a width-validated JSON stimulus interface, Verilator DUT-only line/branch/expression/toggle counters, per-call Responses API traces, raw coverage data, and cumulative cycle/token/time accounting. Yosys pre-process extraction preserves widths and full combinational guards; Rust egg groups proven two-state expressions, with unsupported cases and unmapped counters retained. Clock/reset/state semantics are not folded into combinational classes.

Across both versions, the selected read and write guard classes each cover 12 distinct source locations (six per version). Yosys pre-process extraction retains the activation conditions, type widths, and source spans; a bounded egg run groups equivalent Boolean forms, and independent Z3 checks validate the selected merges. This proves neither four-state SystemVerilog equality nor sequential/reset equivalence. Eighteen unsupported expressions per version are reported, and unmatched counters stay in the raw denominator. Equivalent guards never transfer coverage hits between locations.

The generated actions were replayed through the fixed driver after the live run, and an independent Claude Code Opus 5.5 review was requested in blinded X/Y and then unblinded artifact-audit stages. Claude's blinded pass reproduced all 12 target costs; its unblinded pass found no mechanistic evidence that the guard groups caused C's earlier crossings and flagged the three-valid-item checker boundary. We resolved these findings against raw artifacts and saved a written disposition.

The result bundle records exact source hashes, tool versions, model settings, prompts, actions, failures, coverage files, target costs, and plots. Compilation, initialization, and the separate directed ceiling are recorded apart from arm-specific simulator calls. The final project code will be submitted with the December 18 report.

All 48 GPT-6 Sol calls returned the requested model and passed action validation without retries. They consumed 944,948 API input and 11,774 output tokens. The 48 arm-specific simulations executed 768 cycles in total, including replay. Independent replay matched every one of 48 rounds on 33 stable fields and all 52 raw coverage files byte for byte. Eight checker, feedback-isolation, mapping, and replay tests passed. No checker-detected queue defect occurred in this pilot; it therefore cannot measure bug-detection sensitivity on the live DUT.

The fixed queue checker uses a separate software occupancy and front data model rather than treating the RTL's own outputs as expected values. It tests reset, clear, disabled-cycle hold, empty/full and valid front data. Unit controls include deliberately altered behavior to confirm that the checker reports a real mismatch. The checker models three valid items for DEPTH=4, matching the RTL's internal full threshold. The prose calls DEPTH the number of entries but does not state an independent numeric full capacity; a fourth write is excluded. Almost-full and almost-empty timing in the CVDP prose was too ambiguous for an ordinary-cycle oracle, so those observations are excluded from functional verdicts except their reset/clear state. This contract is narrower than unrestricted use, and zero live failures cannot establish full-spec correctness.

All simulator runs use the same Verilator Coverage-3 build flags and DUT-only source filter. The raw line metric includes emitted case, block, and elsif counters; branch counts separate outcomes; expression counts tool-reported Boolean outcomes. Testbench lines do not enter these denominators. Coverage is flushed even when a checker reports a failure. The run metadata separates model latency, compilation, simulation, and egg preprocessing; the per-run wall cost amortizes shared setup under a stated policy.

IV. NEXT STEPS

For the November 6 midterm, present the measured B/C trajectories, proof-backed guard examples, independent replay, Claude audit, and limitations. For Milestone 2 on November 9, preregister a harder reachable target in at least two independent IPs, validate each specification checker, and add a length-matched placebo appendix. A deeper FIFO/controller should exercise full/empty, simultaneous operations, wraparound, and reset during activity. Add the text-only A arm to quantify raw coverage value separately from semantic feedback; retain matching budget, seeds, tools, and raw denominators.

Before paid runs, screen candidate tasks for multiple independently reachable, source-distinct equivalent uncovered predicates and nested guards; freeze the DUT version, checker, driver, and coverage denominator. Add a known-good control and single-fault controls to test checker sensitivity. For each model call, retain the rendered prompt, returned model ID, API token usage, accepted actions, and error/retry status. Then compare coverage against tokens, cycles, simulator calls, and wall time, including not-reached runs and preprocessing cost.

A concrete Milestone 2 decision gate is: the new DUT must have a reachable target below its directed reference ceiling but above what the common seed and first raw-feedback round usually reach. Its specification must determine unambiguous expected outputs at each tested cycle. Any semantic group must contain multiple source-distinct uncovered locations, full activation guards, proven binary rewrites, and a recorded mapping to raw coverage. A task failing these conditions is a useful infrastructure control but not evidence for or against semantic guidance.

For the midterm presentation, show one exact guard before and after normalization, the untouched source counters it maps to, and four cost curves per RTL version. Explain why the old arbiter and FIFO saturated quickly, why the queue variants are still one IP, and which checker behaviors are independently established. For the November 9 report, include the new task screen, preregistered thresholds, a placebo appendix control, and all individual trials including invalid outputs and misses. The December 18 code release will contain exact scripts, frozen hashes, prompts, and raw counters.

We will declare both a fixed-budget endpoint and a cost-to-target outcome before seeing new results. Fixed-budget reporting includes every attempted model call and every simulator execution. A missed target is marked not reached, not removed from the sample. Token ROI will be newly covered DUT branch points per 1,000 API-reported input-plus-output tokens; cached input and reasoning output are subsets of those totals. We will also report raw new points, line and toggle metrics, checker categories, wall time, and per-round traces. Comparing C with B answers the semantic question; comparing B with A answers the simpler coverage-feedback question.

A second-agent audit remains useful for finding arithmetic, provenance, or interpretation mistakes, but it cannot replace independent simulation. We will blind arm labels for an initial review, then reveal prompt content and ask for source-level challenges. Every finding needs an explicit disposition against frozen logs, raw coverage files, or a separate replay. This avoids treating agreement between language models as functional evidence.

We will also use open-source formal verification tools, including Yosys equivalence checking and SymbiYosys with an SMT solver, to check selected IPs before comparing agents. We will prove RTL variant equivalence under documented reset assumptions and check properties such as occupancy bounds, legal-input overflow and underflow behavior, and reset consistency. Proof results, bounded depths, counterexamples, and unresolved properties will be recorded. Together with specification-based simulation, these checks will strengthen confidence in IP usability and formal consistency.

Finally, scaling to a stateful FIFO/controller requires careful reset semantics and oracle timing. We will preserve the RTL's event context outside combinational e-graph rewrites, test synchronous and asynchronous reset behavior separately, and avoid claiming four-state equivalence from two-state proofs. A design with several independent reachable bottlenecks should leave room for semantic grouping to change the model's stimulus choice. If all arms saturate after one round again, we will report the ceiling and move to a stronger held-out design rather than optimizing the prompt after seeing test outcomes.

The present result neither proves a general E-graph benefit nor rules one out. It tests whether the current semantic feedback helps on one queue IP and its equivalence-checked syntax variant under a fixed stimulus-only verification boundary.

The cited work motivates the benchmark, feedback, and proof method; its published quality metrics are not numerically comparable with this pilot.

REFERENCES

[1] N. Pinckney et al., Comprehensive Verilog Design Problems, arXiv:2506.14074, 2025.

[2] H. Zhang et al., LLM4Cov, arXiv:2602.16953, 2026.

[3] C. G. G. Coward et al., ROVER, arXiv:2406.12421, 2024.

[4] Y. Cai et al., SmoothE, ASPLOS 2025.

[5] Z. Zhang et al., ASPEN, MLCAD 2025.